Ames Housing Price Prediction

A Full Walkthrough of the Machine Learning Pipeline:

What We Did, Why We Did It, and What the Results Show

Dataset: Ames Housing (2,930 houses, 79 predictor features + SalePrice target)

Task type: Supervised Regression | Target: SalePrice ($)

This document is a presentation-ready explanation of the notebook *AMESprojectFINAL_fixed.ipynb*. Every stage of the project is covered: the reasoning behind each decision, the exact code-level choice made, and the actual numeric result produced when the notebook was run.

Overview of the Project

The goal of the project is to build a regression model that predicts a house's sale price (**SalePrice**) from its physical and locational characteristics (size, quality, neighborhood, age, amenities, etc.) using the Ames, Iowa housing dataset (2,930 rows, 79 candidate predictor columns).

The notebook follows a full, disciplined ML workflow: understand the data → explore it → engineer features → split data correctly → build preprocessing pipelines → train multiple candidate models → tune the best ones → evaluate fairly on a held-out test set → interpret the winning model. The sections below follow the notebook's own structure, section by section.

- **Section 1** — Dataset Understanding
- **Section 2** — Exploratory Data Analysis (EDA)
- **Section 3** — Data Analysis & Business Insights
- **Section 4** — Data Preprocessing (the most important section for avoiding mistakes)
- **Section 5** — Pipelines & ColumnTransformer
- **Section 6** — Model Development (baseline models + cross-validation)
- **Section 7** — Hyperparameter Tuning
- **Section 8** — Model Evaluation & Comparison (Train / Validation / Test)
- **Section 9** — Feature Importance & Interpretation
- **Section 10** — Final Model Selection, Overfitting/Underfitting Verdict, and Recommendations

Section 1 — Dataset Understanding

Before touching any model, the raw CSV file was loaded and inspected to understand its size, structure, and quality.

What we did

- Loaded **AmesHousing.csv** and cleaned the column names (removed spaces/slashes, e.g. "Lot Area" → "LotArea") so they behave as valid Python identifiers.
- Dropped **Order** and **PID** — these are just row/parcel identifiers with zero predictive value, and keeping them risks a model "memorizing" an ID instead of learning real patterns.
- Checked data types, duplicate rows, and the amount of missing data per column.

> ***Why we did this:*** *Identifier columns like PID are unique per row, so a model could technically use them to "look up" a price it has memorized from training — this is a classic beginner mistake that inflates training accuracy while being useless (and dangerous) on new houses.*

Result / what we observed: 2,930 rows × 82 columns → 80 columns after dropping identifiers. No duplicate rows. 43 object (text/categorical) columns, 26 integer columns, 11 float columns.

Missing Values

27 out of 80 columns contained missing values. The table below shows the most affected ones:

Column	Missing %	Column	Missing %
PoolQC	99.56%	GarageQual	5.43%
MiscFeature	96.38%	GarageCond	5.43%
Alley	93.24%	GarageYrBlt	5.43%
Fence	80.48%	GarageFinish	5.43%
MasVnrType	60.58%	GarageType	5.36%
FireplaceQu	48.53%	BsmtExposure	2.83%
LotFrontage	16.72%	BsmtQual/Cond/FinType1/2	~2.7% each

Key insight that shaped everything downstream

The data dictionary reveals that for most of these columns, a missing value does **not** mean the data is unknown — it means the feature simply doesn't exist for that house. For example, a missing **PoolQC** means "this house has no pool", a missing **GarageType** means "no garage", a missing **BsmtQual** means "no basement". Only a handful of columns (LotFrontage, MasVnrArea, Electrical, and a few basement/garage numeric fields) are genuinely unknown values that need statistical imputation.

> ***Why we did this:*** *Treating a real "no such feature" case as if it were a random missing value (e.g. filling PoolQC's NaN with the most common pool quality) would fabricate pools that don't exist. This distinction directly drove the preprocessing strategy in Section 4.*

Feature Grouping

Based on the official data dictionary, the 79 predictor columns were split into three groups, because each group needs a different encoding strategy later:

Group	Count	Examples	Why this group is treated differently
Numerical	34	GrLivArea, LotArea, YearBuilt	Continuous measurements → scaled, log-transformed if skewed
Nominal categorical	25	Neighborhood, MSZoning, SaleType	No natural order → one-hot encoded
Ordinal categorical	20	ExterQual, KitchenQual, GarageFinish	Has a natural rank (Po<Fa<TA<Gd<Ex) → encoded 0,1,2,3...

Section 2 — Exploratory Data Analysis (EDA)

2.1 — The target variable, SalePrice

SalePrice was plotted as a histogram, and its skewness was measured.

Result / what we observed: Skewness of raw SalePrice = **1.744** (strongly right-skewed: most homes sell in the $100k–$250k range, but a long tail of expensive homes pulls the distribution). After applying a **log1p** transform, skewness dropped to –**0.015** — essentially symmetric.

> ***Why we did this:*** *Linear models (Linear/Ridge/Lasso Regression) and RMSE-based training implicitly assume roughly normal, homoscedastic (constant-variance) errors. Training directly on a heavily skewed target would let the few very expensive houses dominate the loss function and distort the fit for the vast majority of "normal" houses. The decision made here —* ***train on log1p(SalePrice), and always invert with expm1() before reporting metrics in dollars*** *— is used consistently for the rest of the notebook.*

2.2 — Relationship between GrLivArea and SalePrice, and the outlier discovery

A scatterplot of GrLivArea (above-ground living area) against SalePrice revealed a strong positive relationship (correlation = **0.707**), but also 3 unusual houses with very large living area (>4,000 sq.ft.) that sold for surprisingly *low* prices (below $300k).

> ***Why we did this:*** *This is a well-documented quirk of the Ames dataset — the dataset's original author (De Cock) explicitly recommends removing these rows, since they are almost certainly partial/incomplete or distressed sales that don't reflect normal market behavior. Left in place, they would pull the fitted line away from the true relationship, especially for linear models and SVR which are sensitive to a few extreme points. (Exactly how and when these rows were removed — without causing leakage — is explained in Section 4.)*

2.3 — Correlation heatmap

A correlation heatmap of the top numerical features against SalePrice was produced to identify the strongest linear predictors up front.

Feature	Correlation with SalePrice
OverallQual	0.799
GrLivArea	0.707
GarageCars	0.648
GarageArea	0.640
TotalBsmtSF	0.632
1stFlrSF	0.622
YearBuilt	0.558
FullBath	0.546
YearRemodAdd	0.533

Result / what we observed: OverallQual (overall material/finish rating, 1–10) is the single strongest numerical predictor of price — stronger even than living area. This finding is confirmed again later by the Ridge model's coefficients in Section 9.

Section 3 — Data Analysis & Business Insights

Before modeling, a set of concrete business questions was answered directly from the data — this builds domain intuition that later explains *why* the model behaves the way it does.

Price statistics & the biggest price drivers

Statistic	Value
Mean	$180,796
Median	$160,000
Min	$12,789
Max	$755,000
Std. deviation	$79,887

Mean > median confirms the right-skew already seen in Section 2. The cheapest house ($12,789, OldTown) and most expensive ($755,000, NoRidge) differ by **59×**.

Neighborhood effect

The 3 most expensive neighborhoods average roughly **$325,522** combined, versus the 3 cheapest averaging **$101,706** — over a 3× gap purely from location.

Top 3 (highest avg price)	Avg Price	Bottom 3 (lowest avg price)	Avg Price
NoRidge (71 houses)	$330,319	MeadowV (37 houses)	$95,756
StoneBr (51 houses)	$324,229	IDOTRR (93 houses)	$103,753
NridgHt (166 houses)	$322,018	BrDale (30 houses)	$105,608

A one-way **ANOVA test** confirmed this is statistically real, not random noise: F-statistic = 144.4, p-value ≈ 0.00 (i.e. $p < 0.001$), and **Eta-squared = 0.573** — meaning Neighborhood alone explains about 57% of the variance in SalePrice.

> ***Why we did this:*** *This test matters because it's the formal, statistical justification for treating Neighborhood as an important categorical feature in the model, rather than relying on visual impressions alone.*

Other confirmed relationships

- **OverallQual**: correlation 0.80 — average price rises from $48,725 (quality=1) to $450,217 (quality=10).
- **GarageCars**: correlation 0.65 — 3-car garages average $310,305 vs. $104,949 for no garage.
- **FullBath**: correlation 0.55.
- **Basement presence**: houses with a basement average $75,659 more than those without.
- **Kitchen quality**: swings the average price by $231,432 between "Excellent" and "Fair".
- **Central air conditioning**: adds $84,563 on average.
- **House age**: correlation −0.56 (newer houses sell for more, as expected).
- **Bedrooms alone**: a weak correlation (0.14) — bedroom count by itself is a poor predictor, since it doesn't capture the quality/size context a room sits in.

These findings are not just descriptive — several of them (Neighborhood, OverallQual, GrLivArea, SaleCondition) reappear later as the model's strongest learned coefficients in Section 9, giving independent confirmation that the model "learned" real, sensible patterns.

Section 4 — Data Preprocessing

This is the section where most student projects on this exact dataset go wrong, and where this notebook was explicitly rewritten ("_fixed") to correct earlier mistakes. The central theme of this section is: **every preprocessing decision that could "see" the target or the full dataset must be made using ONLY the training data.**

4.1 — Feature Engineering (done first, safely)

Four new features were engineered from existing columns:

- **TotalSF** = TotalBsmtSF + 1stFlrSF + 2ndFlrSF (total finished square footage)
- **HouseAge** = YrSold – YearBuilt
- **RemodAge** = YrSold – YearRemodAdd
- **TotalBaths** = FullBath + 0.5×HalfBath + BsmtFullBath + 0.5×BsmtHalfBath

> ***Why we did this:*** *These are all **row-wise, deterministic** calculations — each new value only uses other columns from the same row. No dataset-wide statistic (like a mean or a fitted parameter) is involved, so it is completely safe to compute these before the train/test split, unlike operations such as scaling or imputation which must be fit on training data only.*

4.2 — Train / Validation / Test Split (moved to happen FIRST)

The dataset was split **60% train / 20% validation / 20% test** using a two-step train_test_split (80/20, then 75/25 of the 80%).

Split	Rows	% of total
Train	1,758	60.0%
Validation	586	20.0%
Test	586	20.0%

> ***Why we did this: This split was deliberately moved to happen BEFORE outlier removal and BEFORE deciding which columns to log-transform.*** *In an earlier (broken) version of this project, both of those steps were done on the full dataset before splitting — meaning the validation and test sets were effectively "cleaned" using information (including the target price, to detect the outliers) that a model is never supposed to see in advance. This is a textbook example of **data leakage**: it makes the validation/test scores look artificially better than what the model would actually achieve on truly unseen houses. Splitting first and applying every subsequent rule using train-only statistics is the fix.*

4.3 — Outlier removal (train set only)

The 2 documented GrLivArea outliers (>4,000 sq.ft. and price < $300k) found in Section 2 were removed — but only from the training set.

Result / what we observed: 2 rows removed from the training set. Validation and test sets keep any such points, since that mirrors the real, unfiltered situation the model would face in production.

> ***Why we did this:*** *Removing outliers from validation/test as well would mean we're only ever testing the model on "easy", pre-cleaned houses — flattering the reported accuracy. Removing them from train only, while leaving val/test untouched, gives an honest test of generalization (and, as we'll see in Section 8, it does create a small but expected validation→test performance gap).*

4.4 — Log-transforming skewed numerical features

18 numerical features other than the target were also right-skewed. The rule used: log1p-transform any feature with |skewness| > 0.75, computed using **training data only**, excluding count/rating/year columns (OverallQual, YearBuilt, FullBath, etc.) where a log transform makes no sense.

Feature	Skew	Feature	Skew
MiscVal	19.47	OpenPorchSF	2.77
PoolArea	17.23	MasVnrArea	2.42
LotArea	14.76	WoodDeckSF	1.58
3SsnPorch	11.29	GrLivArea	1.01
LowQualFinSF	11.04	BsmtUnfSF	0.94
EnclosedPorch	4.35	1stFlrSF	0.91
BsmtFinSF2	4.16	2ndFlrSF / LotFrontage	0.84 / 0.80
ScreenPorch	3.59	TotalSF / BsmtFinSF1	0.78 / 0.76

Why we did this: *Just like SalePrice itself, these skewed predictors (dominated by many zeros and a few huge values, e.g. LotArea, PoolArea) would otherwise let a handful of extreme houses dominate distance-based models (KNN, SVR) and destabilize linear model coefficients. The column list was fixed using train data, then the same fixed list applied identically to val/test — the log1p operation itself needs no fitted parameters, so applying it afterward is leakage-safe.*

Section 5 — Pipelines & ColumnTransformer

Three separate preprocessing pipelines were built — one per feature group identified in Section 1 — and combined with ColumnTransformer.

Pipeline	Steps	Applied to
Numerical	Median imputation → StandardScaler	38 columns (34 original + 4 engineered)
Ordinal	Constant imputation → per-column OrdinalEncoder (custom rank order) → StandardScaler	20 columns
Nominal	Constant imputation → OneHotEncoder(handle_unknown='ignore')	25 columns

Why each choice was made

- **Median imputation for numerical features** — robust to the outliers/skew already noted, unlike mean imputation.
- **Per-column ordinal categories** (not one generic scale for every ordinal column) — columns like LotShape, Functional, Fence use completely different category sets than the standard Po/Fa/TA/Gd/Ex quality scale. Reusing one generic list would silently mis-encode most of them.
- **StandardScaler over MinMaxScaler** — SVR and KNN are distance-based, so without scaling, large-magnitude features like LotArea would dominate the distance metric over small-scale features like OverallQual (1–10). Ridge/Lasso regularization is also scale-sensitive. StandardScaler is more robust than MinMaxScaler to the outliers still present in area-based columns even after the log transform.
- **One-hot encoding is NOT scaled** — its values are already 0/1, so scaling them would distort the encoding.
- **handle_unknown='ignore' / unknown_value=-1** — protects the pipeline from crashing if a rare category appears in validation/test that wasn't seen in training.

Why we did this: *Wrapping every step inside a scikit-learn Pipeline, rather than manually calling .fit_transform() on the whole dataset, guarantees that when cross-validation later splits the training data into folds, each fold's imputer/scaler/encoder is fit ONLY on that fold's training portion — this is what makes the cross-validation scores in Section 6 leakage-free.*

Section 6 — Model Development (Baselines + Cross-Validation)

Five baseline regression algorithms, each with a plausible chance of doing well on tabular housing data, were trained and compared with 5-fold cross-validation on the training set only.

Model	Why it was included
Linear Regression	Simplest possible baseline — every other model must beat this to be worth using.
Ridge Regression	Linear + L2 penalty — expected to handle the many one-hot columns better than plain linear regression.
Lasso Regression	Linear + L1 penalty — can zero out irrelevant one-hot categories entirely.
KNN Regressor	A non-parametric, distance-based alternative with no linearity assumption.
SVR (RBF kernel)	Can capture non-linear relationships between features and price.

Cross-validation results (RMSE on log scale, lower is better)

Model	Mean CV RMSE (log)	Std. dev
Linear Regression	**0.1393**	**±0.0172**
Ridge Regression	0.1204	±0.0121
Lasso Regression	0.1231	±0.0117
KNN Regressor	0.1744	±0.0080
SVR	0.1537	±0.0162

Result / what we observed: Ridge Regression had the lowest (best) mean cross-validation error, closely followed by Lasso. **KNN was clearly the weakest** — a first hint of underfitting that is confirmed later. This is why Ridge and SVR (as the best linear and best non-linear candidates) were chosen for hyperparameter tuning in Section 7, instead of tuning all five models.

> ***Why we did this:*** *5-fold cross-validation (rather than a single train/validation check) was used here specifically to pick which models deserve tuning, because a single split can be lucky/unlucky — averaging over 5 folds gives a more reliable ranking before spending compute on GridSearchCV.*

Section 7 — Hyperparameter Tuning

GridSearchCV (5-fold CV, matching Section 6) was used to tune only the two strongest candidates: **Ridge Regression** and **SVR**.

> ***Why we did this:*** *Linear Regression has no hyperparameters to tune. Lasso and KNN were already clearly behind Ridge/SVR in cross-validation, so tuning them further would not have changed the final choice — tuning was focused where it could actually matter.*

Model	Grid searched	Best parameters found
Ridge Regression	alpha ∈ {0.01, 0.1, 1, 10, 20, 50}	alpha = 20.0
SVR	C ∈ {0.1,1,10,100}, gamma ∈ {scale,0.01,0.1}, kernel = rbf	C = 1, gamma = 0.01, kernel = rbf

What the winning hyperparameters mean

- **Ridge's alpha=20** (a fairly strong L2 penalty) winning suggests the model benefits from shrinking coefficients more aggressively — sensible given how many one-hot-encoded columns feed into the model, since heavier regularization protects against unstable coefficients on rare categories.
- **SVR's C=1** (a relatively small value, out of 0.1–100 tested) means a wider margin / simpler model generalized better than an aggressively tight fit — a small hint that SVR was prone to overfitting at higher C.
- **gamma=0.01** (a small value) means each training point's influence reaches further, i.e. a smoother, less locally-wiggly decision surface — again favoring a simpler model over an overly flexible one.

Result / what we observed: Validation RMSE (log scale): Ridge = **0.1062**, SVR = **0.1290**. **Ridge Regression was selected** as the best-performing tuned candidate based on the validation set — the test set was not looked at during this decision.

> ***Why we did this:*** *The decisive rule followed throughout the whole project: model selection (which model, which hyperparameters) is only ever allowed to use cross-validation and the validation set. The test set is reserved for one single, final, honest check — done in Section 8.*

Section 8 — Model Evaluation & Comparison

All 5 baseline models plus the 2 tuned candidates were evaluated on Train / Validation / Test, with metrics computed on the original dollar scale (predictions inverse-transformed with expm1).

Model	Split	MAE ($)	RMSE ($)	R2
Linear Regression	Train	11,851	17,133	0.9519
Linear Regression	Validation	13,924	19,285	0.9322
Linear Regression	Test	14,632	28,218	0.9007
Ridge Regression	Train	12,319	18,103	0.9463
Ridge Regression	Validation	13,613	18,564	0.9372
Ridge Regression	Test	14,670	30,455	0.8843
Lasso Regression	Train	13,291	19,979	0.9346
Lasso Regression	Validation	13,599	18,604	0.9369
Lasso Regression	Test	15,215	31,358	0.8774
KNN Regressor	Train	16,505	28,066	0.8710
KNN Regressor	Validation	19,065	27,912	0.8580
KNN Regressor	Test	21,580	37,128	0.8281
SVR	Train	11,087	14,464	0.9657
SVR	Validation	14,351	21,325	0.9171
SVR	Test	15,787	28,309	0.9000
Ridge (tuned)	**Train**	**12,490**	**18,491**	**0.9440**
Ridge (tuned)	Validation	13,626	18,561	0.9372
Ridge (tuned)	Test	14,719	30,726	0.8822
SVR (tuned)	Train	11,063	14,438	0.9659
SVR (tuned)	Validation	14,074	20,701	0.9219
SVR (tuned)	Test	15,170	27,774	0.9038

Reading the table: overfitting vs. underfitting, model by model

- **KNN Regressor — underfitting.** It is the weakest model on *every single split* (train R^2=0.871, validation=0.858, test=0.828) — the gap between train and validation is small, which rules out overfitting; it simply never fits the data well to begin with. This is likely because plain Euclidean-distance KNN struggles in the ~250-dimension, mostly one-hot-encoded feature space produced after preprocessing.
- **SVR (untuned) — mild overfitting.** Train R^2=0.966 vs. Validation R^2=0.917 — a real ~5-point gap. After tuning (C=1, gamma=0.01, favoring a simpler model), the gap narrows slightly (train 0.966 / validation 0.922), showing the tuning did what it was meant to do.
- **Ridge and Linear Regression — minimal overfitting.** Train-to-validation R^2 gaps are small (within about 1–2 points for Ridge, e.g. 0.944→0.937 tuned), which is a healthy sign.
- **Validation → Test gap (the real story here).** Every model's test R^2 drops noticeably below its validation R^2 (e.g. Ridge tuned: 0.937 → 0.882). This is *expected and correctly explained* by the

preprocessing decision in Section 4: the 2 documented GrLivArea outliers were removed only from the training set, so validation and test still contain a few unusually hard, high-leverage houses. Ridge's squared-error loss is more sensitive to these few large errors than SVR's margin-based loss — which is exactly why SVR (tuned) ends up slightly ahead of Ridge on the test set specifically (test R^2=0.9038 vs. 0.8822), despite Ridge having won on validation.

Bottom line: after the leakage fixes in Section 4, none of the models show the kind of large, uncontrolled train-validation gap that would signal serious overfitting. The one real underfitting case is KNN. The validation→test drop seen across all models is a deliberate, explained consequence of testing on realistically "messy", un-cleaned data — not a bug.

Section 9 — Feature Importance & Interpretation

Because the selected model (Ridge Regression) is linear, its fitted coefficients on the standardized, log-transformed feature space directly indicate each feature's estimated effect on log(SalePrice) — larger absolute coefficients mean a bigger estimated price impact, holding other features fixed. (Had a non-linear model like SVR been selected instead, permutation importance — already implemented in the notebook as a fallback — would have been used instead, since SVR with an RBF kernel has no single interpretable coefficient per feature.)

What came out on top, and how it connects back to earlier sections

- **OverallQual** and **GrLivArea** are, once again, the two strongest predictors — consistent with the correlation heatmap (Section 2) and the group-by analyses (Section 3).
- **TotalSF** (the engineered feature) ranks 4th, ahead of any of its individual components (TotalBsmtSF, 1stFlrSF, 2ndFlrSF) — a good sign that combining the three size features in Section 4 captured real signal, not just redundant noise.
- **SaleCondition** categories (Abnormal = negative, Partial/Normal = positive) matter almost as much as square footage — i.e. *how* a house was sold (foreclosure/short sale vs. a newly-built home sold before completion vs. a normal sale) is a genuinely new finding not examined directly in the earlier EDA.
- **Neighborhood_Crawfor** (positive) and **Neighborhood_IDOTRR** (negative) echo the neighborhood price gaps found in Section 3, and the ANOVA test's conclusion that neighborhood has a statistically significant relationship with price.
- A few **Exterior1st** and **MSZoning** categories also appear (e.g. brick-face siding positive, asbestos-shingle siding negative, commercial zoning negative) — plausible, though each represents a smaller sample than the top numeric features, so these should be read as suggestive rather than as strong as OverallQual/GrLivArea.

Caveat worth stating out loud in the presentation: Ridge coefficients estimate each feature's effect *holding all other features fixed* — they describe association within this dataset and this specific model, not necessarily a provable causal effect.

Section 10 — Final Model Selection & Conclusions

Final model: Ridge Regression (tuned, alpha = 20)

Split	R2	MAE	RMSE
Train	0.9440	$12,490	$18,491
Validation	0.9372	$13,626	$18,561
Test	0.8822	$14,719	$30,726

Ridge Regression (tuned) was selected as the final model, following the project's protocol of choosing based on cross-validation and validation performance only — it achieved the best validation R^2 among the tuned candidates, with a small, healthy train-validation gap indicating limited overfitting. It was preferred over the other linear models because its L2 regularization helps control coefficients in the high-dimensional, mostly one-hot-encoded feature space, and over SVR/KNN because it is directly interpretable (Section 9) while remaining competitive.

Honest caveat to mention to the professor

The one-time test evaluation shows that **SVR (tuned)** actually reaches a slightly higher test R^2 (0.9038) and lower test RMSE ($27,774) than the selected Ridge model (0.8822 / $30,726). This does **not** make the selection process wrong — the choice was correctly made blind to the test set, following proper ML methodology — but it shows that Ridge's advantage on validation didn't fully carry over to test, most likely because the few high-leverage outlier houses left in validation/test (after the Section 4 leakage fix) hurt Ridge's squared-error loss more than SVR's margin-based loss. Ridge remains the reported final model per protocol, with SVR (tuned) flagged as a strong alternative worth revisiting.

Overfitting / Underfitting — final verdict

- **No serious overfitting** in the final model: Ridge's train→validation R^2 gap is tiny (0.944 → 0.937).
- **Mild overfitting** was observed in untuned SVR (train 0.966 vs. val 0.917), and tuning reduced it, as intended.
- **Underfitting** was clearly observed in KNN — consistently the weakest model on every split.
- **No data leakage** — this was the main risk in this specific project (the dataset combines a moderately small size with many features), and it was addressed head-on: the train/val/test split happens before any decision (outlier removal, skew detection) that could peek at validation/test data.

Limitations & recommended next steps (for future work / to mention as discussion points)

- **Try tree-based ensembles** (Random Forest, Gradient Boosting / XGBoost / LightGBM / CatBoost). Ames Housing has non-linear feature interactions (e.g. the effect of OverallQual changes depending on Neighborhood) that a linear model like Ridge cannot capture without manual interaction terms, while tree ensembles capture such interactions automatically and are also more robust to the remaining outliers.
- **Try stacking/ensembling** Ridge (good on the linear part of the signal) with a Gradient Boosting model (good on the non-linear part) — this combination typically outperforms any single model alone on this exact dataset.

- **Revisit the validation→test gap** — since it stems from outliers being present only in val/test, a robustness check (e.g. Huber loss, or reporting median absolute error alongside RMSE) would show whether the gap shrinks under a metric less sensitive to a few extreme errors.
- The EDA in Sections 2–3 (correlations, ANOVA, the GrLivArea outlier threshold) was computed on the **full** dataset before the split — this is standard exploratory practice and did not leak into any fitted model parameter, but it is worth acknowledging as a soft methodological note: in an extremely strict "zero peeking" setup, exploratory statistics would also be computed on a held-out exploration sample only.

Prepared as a presentation companion to AMESprojectFINAL_fixed.ipynb — all figures above are the notebook's actual saved outputs, not estimates.